Name :Laasya Chaitanya Reddy

Roll no:AM.SC.U4AIE23125

PL-SQL Lab 1

1. Do all the questions discussed in the class

1) salary update function (10% hike if dept avg > emp salary)

```
create or replace function sal_update(empno emp.eno%type) returns void as
$$
declare
salary emp.sal%type;
avg_sal emp.sal%type;
begin
select sal into salary from emp where eno = empno;

select avg(sal) into avg_sal
from emp
where dno = (select dno from emp where eno = empno);

if salary < avg_sal then
    update emp
    set sal = sal + sal * 0.1
    where eno = empno;
end if;

end;
$$
language 'plpgsql';
```

2) function to return department name given an employee number

```
create or replace function name(empno emp.eno%type) returns dept.dname%type as
$$
declare
deptname dept.dname%type;
begin
select dname into deptname
from dept
where dptno = (select dno from emp where eno = empno);

return deptname;
end;
$$
language 'plpgsql';
```

2. Consider the following schema .
   Emp(eno, ename, sal)
   EMP_PROJ(eno, pno, prjt_hrs).

   a) Write a function ProjectLoad that returns the total project working hours for the given eno.
   b)
   c) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

      New salary is current sal + difference between currrent sal and average salary .

```
create or replace function project_load(p_eno emp.eno%type) returns int as
$$
declare
total_working_hrs int := 0;
begin
select sum(prjt_hrs) into total_working_hrs from emp_proj where eno = p_eno;
return total_working_hrs;
end;
$$
Language 'plpgsql'

create or replace function sal_update(emp_no emp.eno%type) returns void as
$$
declare
avg_sal int;
begin
select avg(sal) into avg_sal from emp;
update emp set sal = sal +(avg_sal - sal) where sal<avg_sal;
end;
$$
Language 'plpgsql'
```

3. Consider the following tables
   Item(ino, iname, unit_price)
   Transaction(tr_no,ino,qty)

   Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
create or replace function item(item_no Item.ino%type) returns void as
$$
declare
txn_count :=0;
begin
select count(*) into txn_count from Transaction_tab where ino = item_no;
if txn_count <2 then
delete from item where ino = item_no;
end if;
end;
$$
Language 'plpgsql'
```

4.

Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)
BRANCHES(br_no, br_name, loc)
CUSTOMER(cno, cname, c_type)

a) Write a function that accepts a threshold value and a customer number . The program updates the c_type based on the threshold value. Ie. If balance> threshold then class A, else class B.
b) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.
c) Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```
create or replace function update_customer_type(threshold numeric, custo_no ACCOUNTS.cust_no%type )
returns void as
$$
declare
balance ;
begin
select bal into balance from ACCOUNTS where cust_no = custo_num;

if balance > threshold then
update CUSTOMER set c_type = 'a' where cno = custo_no;
else
update CUSTOMER set c_type = 'b' where cno = custo_no;
end if;

end;
$$
Language 'plpgsql'
```

```
create or replace function closebranch(p_close_br ACCOUNTS.br_no%type, p_new_br ACCOUNTS.br_no%type)
returns void as
$$
begin
update accounts set br_no = p_new_br where br_no = p_close_br;

delete from branches where br_no = p_close_br;

end;
$$
language 'plpgsql';
```

```sql
create or replace function safe_withdraw(p_ac_no int, p_amount numeric)
returns text as
$$
declare
current_bal numeric := 0;
begin
select bal into current_bal from accounts where ac_no = p_ac_no;

if current_bal < p_amount then
    return 'insufficient funds';
else
    update accounts set bal = bal - p_amount where ac_no = p_ac_no;
    return 'withdrawal successful';
end if;

end;
$$
language plpgsql;
```